

山から粒子が落ちる

コール図

main 関数 (wWinMain) から呼ばれる関数と、その役割

対象: water_from_mountain (C++ / Direct3D 11 / HLSL)

https://github.com/githubtarohario/water_from_mountain

各関数の役割はソースの「// 概要 :」コメントから自動抽出しています

図の見方

本書の図は、関数を箱で表し、矢印で「どの関数がどの関数を呼ぶか」を示したコール図です。箱の上段が関数名、下段がその関数の役割です。役割の文はソースコードの「// 概要 :」コメントから自動で抜き出しているため、実装と一致しています（全文は巻末の付表を参照）。

 main.cpp  Graphics  Terrain  SPH  FluidRenderer  Camera  Noise / Loader / Win32 / D3D  HLSL (GPU)

表記	意味
箱の左の色帯	その関数が属するモジュール（上の凡例の色）
実線の矢印	直接の呼び出し（呼ぶ側 → 呼ばれる側）
破線の矢印	別の経路からも呼ばれる共通処理
点線の枠	処理のまとまり（分岐する系統、並列実行される範囲など）
の印	OpenMP により粒子ごとに並列実行される関数
オレンジの箱	GPU 上で動く HLSL シェーダーの関数

図は 6 枚あります。図 1 が全体像で、そこから初期化（図 2・3）、メインループ（図 4）、物理（図 5）、描画（図 6）へと掘り下げていきます。

図 1 全体のコール図 — wWinMain から見た呼び出し

箱 = 関数（上段: 関数名、下段: 役割）。矢印 = 呼び出し。色 = 所属モジュール

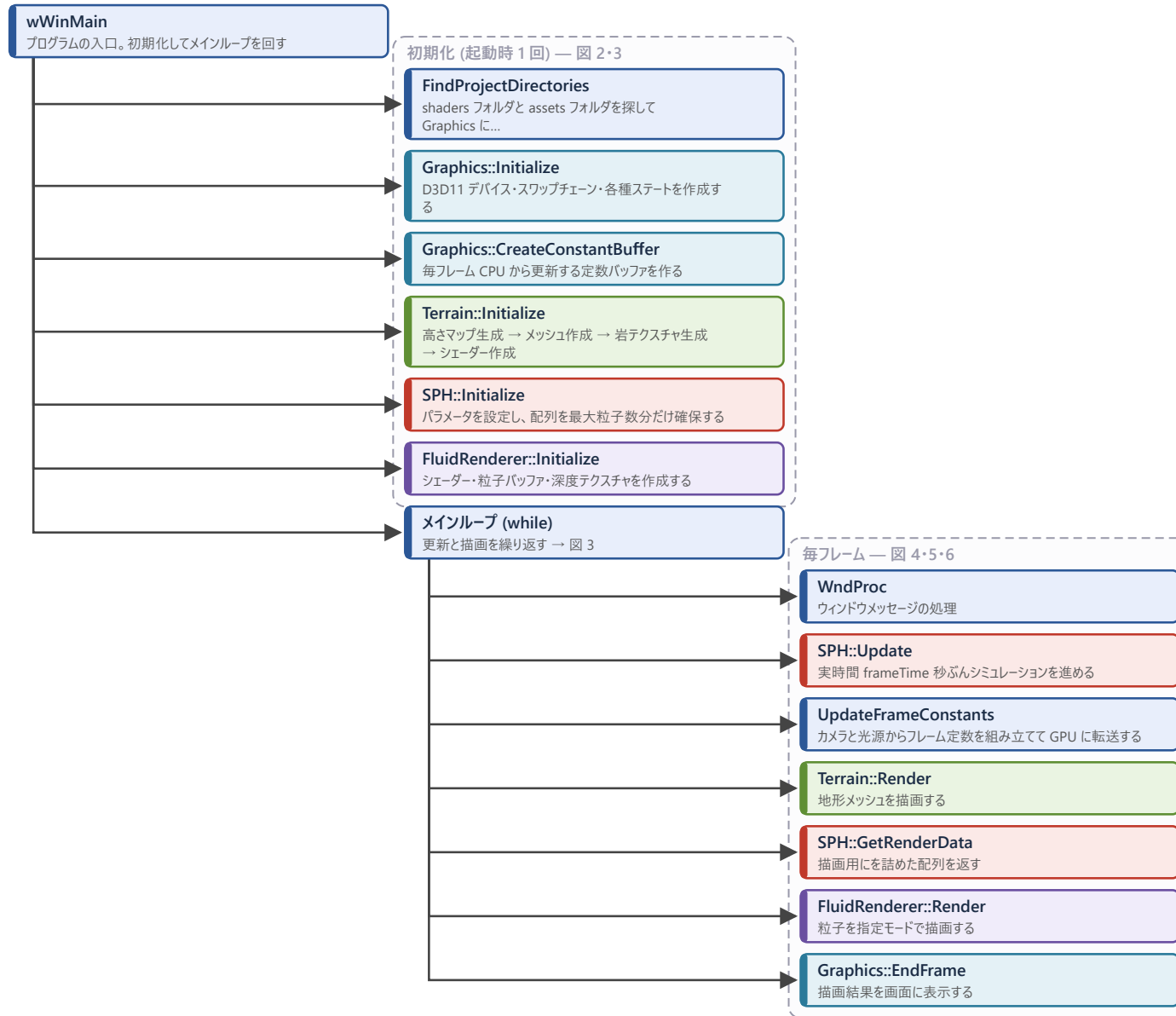


図 2 初期化のコール図 (Graphics / SPH / FluidRenderer)

実線 = 直接の呼び出し。Terrain::Initialize の内部は図 3

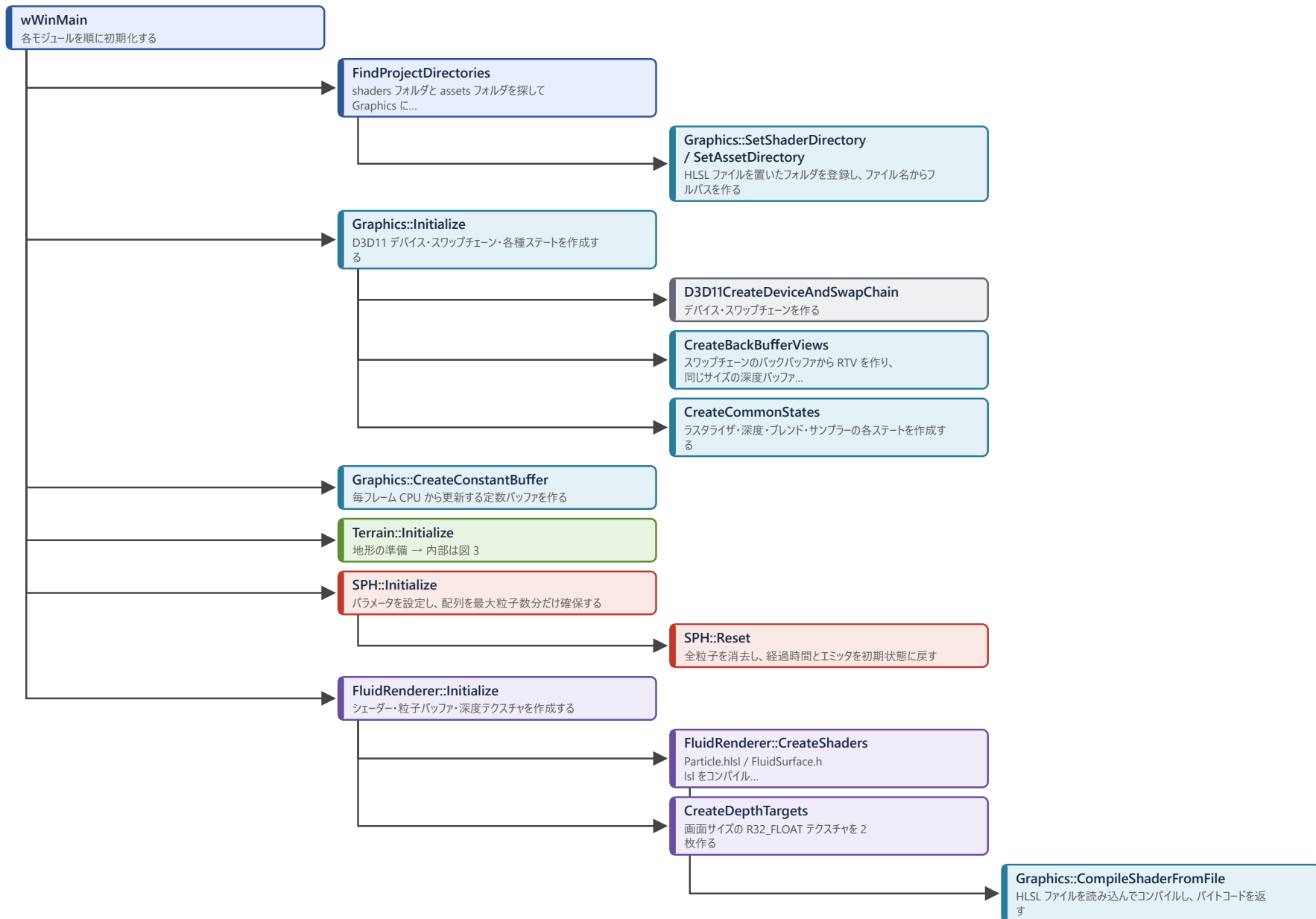


図 3 Terrain::Initialize のコール図

地形は A / B のどちらか一方を通る。破線 = 両方から呼ばれる共通処理

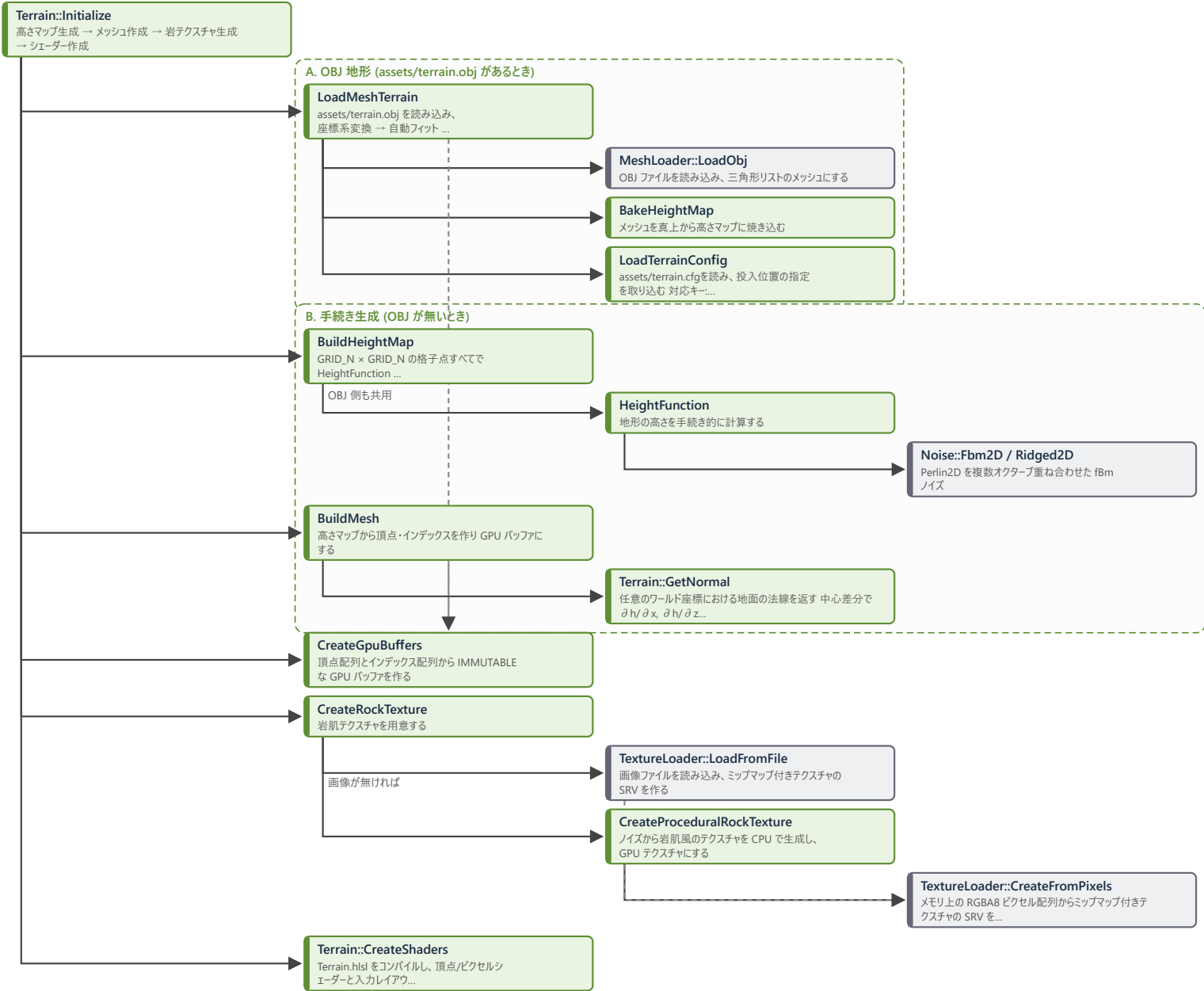


図 4 メインループ 1 フレームのコール図

上から下が実行順。オレンジの箱は GPU 上で動くシェーダー

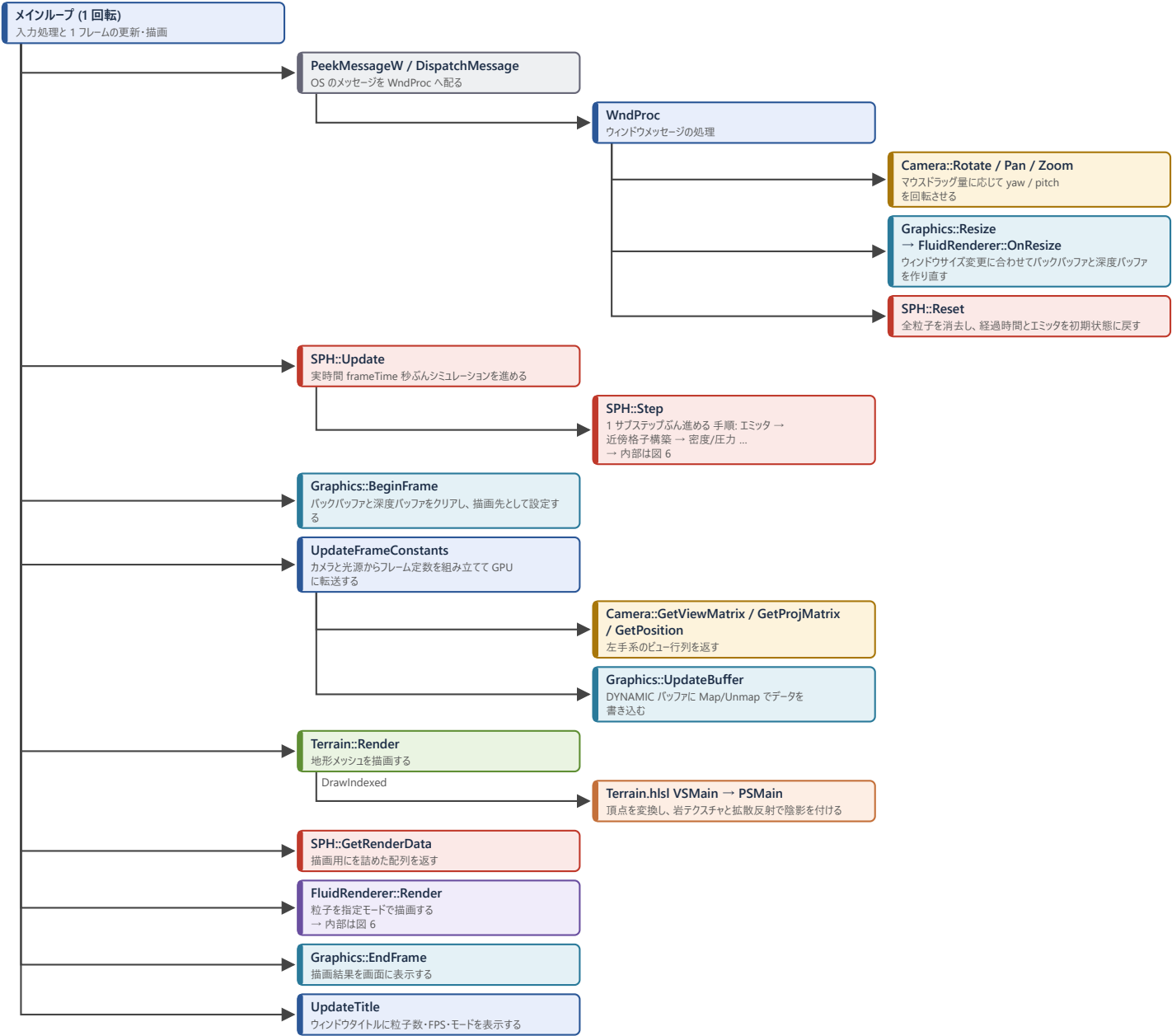


図 5 SPH（物理シミュレーション）のコール図

1 サブステップの中で上から順に呼ばれる。Terrain へは高さ・法線の問い合わせだけを行う

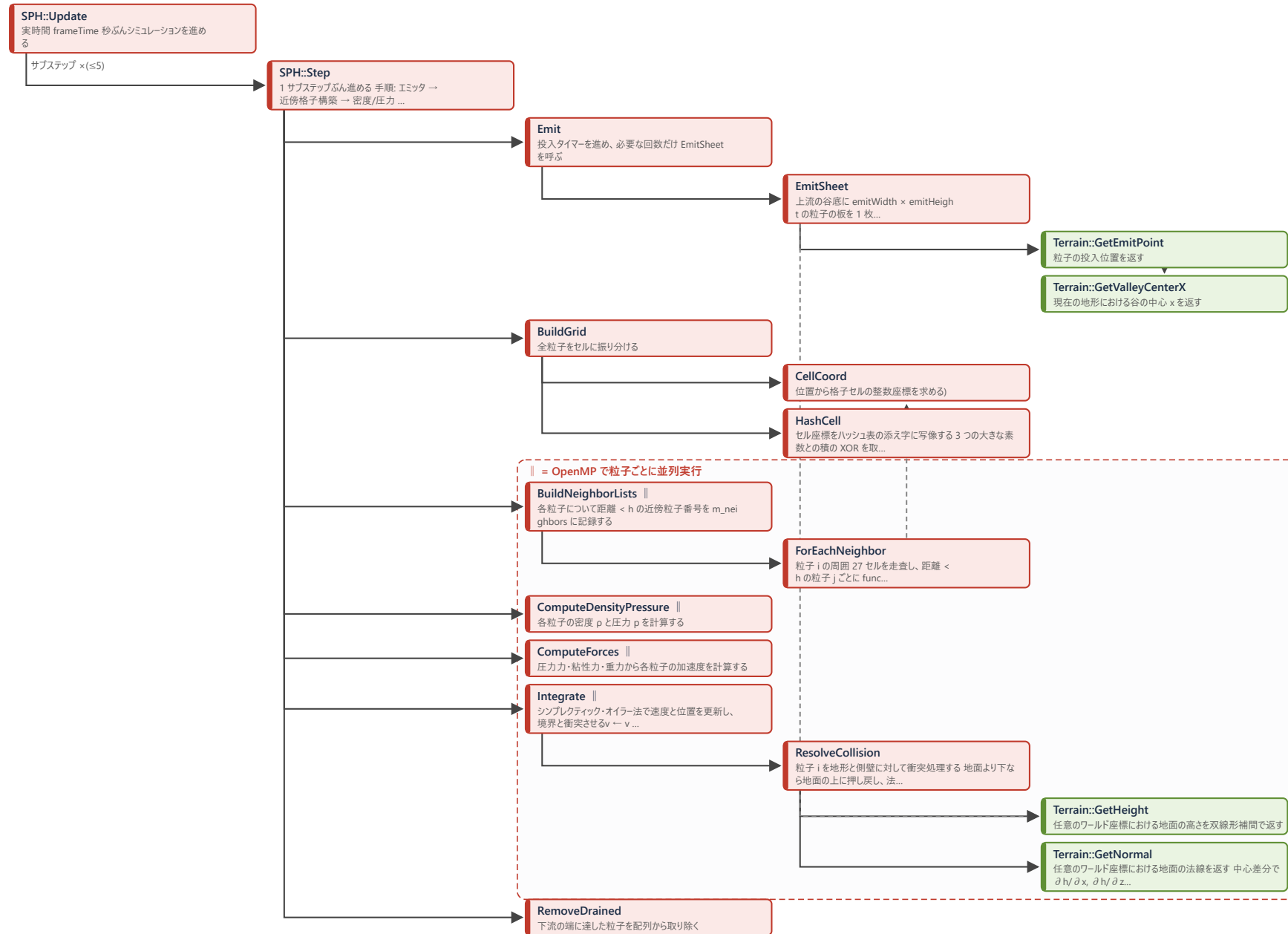
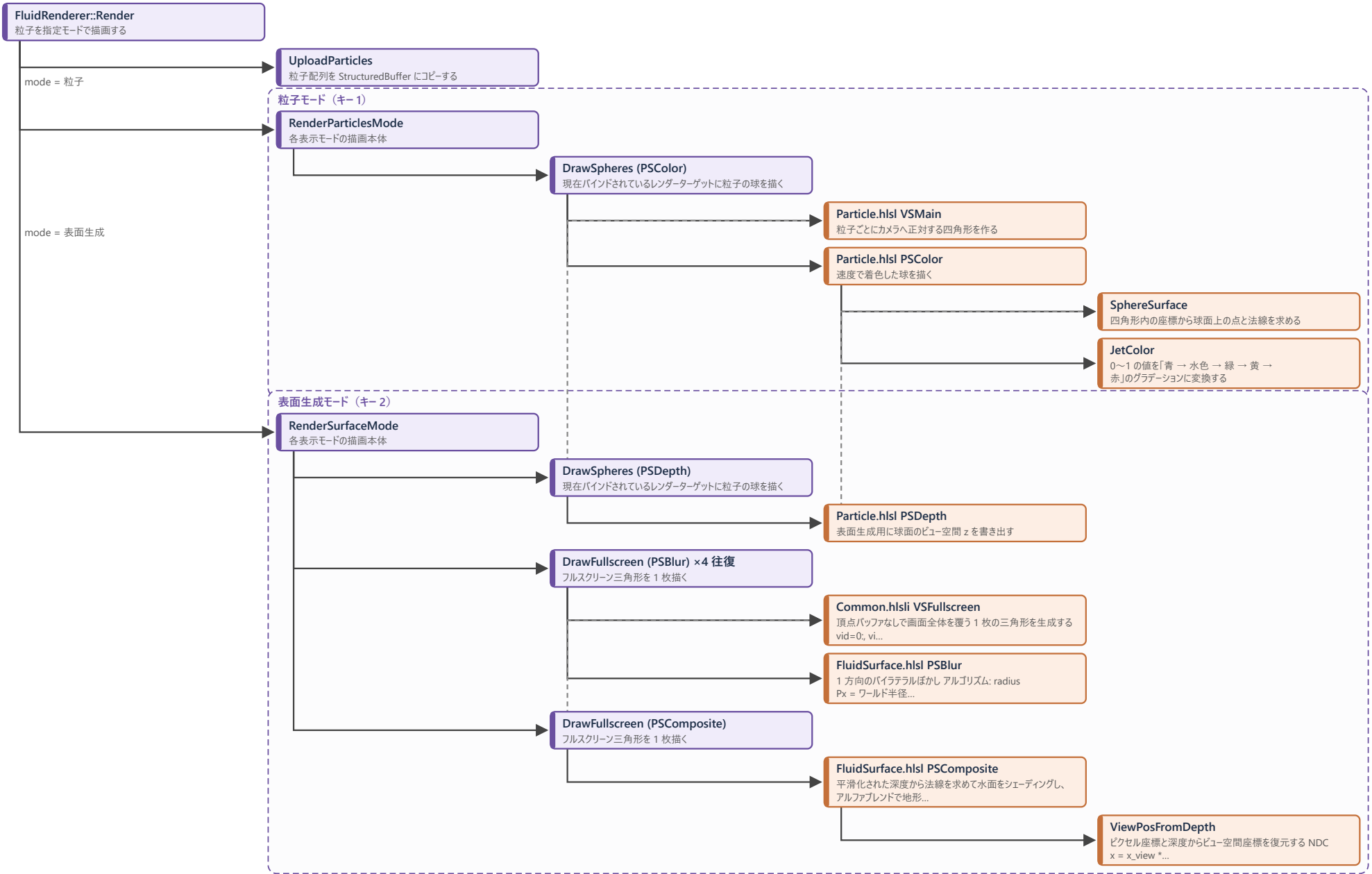


図 6 描画（FluidRenderer）のコール図

オレンジの箱は GPU 上のシェーダー関数。DrawSpheres / DrawFullscreen が描画命令を出すと呼ばれる



付表 関数一覧と役割

図に登場する関数の役割を一覧にしたものです（ソースの「// 概要 :」コメントより自動生成）。

関数	定義場所	役割
<code>main::wWinMain</code>	main.cpp	ウィンドウ作成 → 各モジュール初期化 → メインループ
<code>main::FindProjectDirectories</code>	main.cpp	shaders フォルダと assets フォルダを探して Graphics に登録する。実行ファイルの場所が bin¥ や bin¥Release¥ でも動くように、カレントディレクトリ → exe のフォルダ → その親 → さらに親 の順に探す。(shaders¥Common.hlsl の存在で判定し、assets は同じ階層とみなす)
<code>main::UpdateFrameConstants</code>	main.cpp	カメラと光源からフレーム定数を組み立てて GPU に転送する
<code>main::UpdateTitle</code>	main.cpp	ウィンドウタイトルに粒子数・FPS・モードを表示する
<code>main::WndProc</code>	main.cpp	ウィンドウメッセージの処理 (入力・リサイズ・終了)
<code>main::FileExists</code>	main.cpp	ファイルが存在するか調べる
<code>Graphics::Initialize</code>	Graphics	D3D11 デバイス・スワップチェーン・各種ステートを作成する
<code>Graphics::CreateBackBufferViews</code>	Graphics	スワップチェーンのバックバッファから RTV を作り、同じサイズの深度バッファ (D24S8) と DSV を作る
<code>Graphics::CreateCommonStates</code>	Graphics	ラスタライザ・深度・ブレンド・サンプラーの各ステートを作成する
<code>Graphics::Resize</code>	Graphics	ウィンドウサイズ変更に合わせてバックバッファと深度バッファを作り直す
<code>Graphics::BeginFrame</code>	Graphics	バックバッファと深度バッファをクリアし、描画先として設定する
<code>Graphics::EndFrame</code>	Graphics	描画結果を画面に表示する (Present)
<code>Graphics::CompileShaderFromFile</code>	Graphics	HLSL ファイルを読み込んでコンパイルし、バイトコードを返す
<code>Graphics::SetShaderDirectory</code>	Graphics	HLSL ファイルを置いたフォルダを登録し、ファイル名からフルパスを作る
<code>Graphics::CreateConstantBuffer</code>	Graphics	毎フレーム CPU から更新する定数バッファ (DYNAMIC) を作る
<code>Graphics::UpdateBuffer</code>	Graphics	DYNAMIC バッファに Map/Unmap でデータを書き込む
<code>Terrain::Initialize</code>	Terrain	高さマップ生成 → メッシュ作成 → 岩テクスチャ生成 → シェーダー作成
<code>Terrain::LoadMeshTerrain</code>	Terrain	assets/terrain.obj を読み込み、座標系変換 → 自動フィット → 高さマップ焼き込み → GPU バッファ作成 を行う 座標系変換: Blender の OBJ 出力 (右手系, Forward -Z / Up Y) を DirectX の左手系に合わせるため x を反転する 自動フィット: xz の大きい方の辺が WORLD_SIZE になるよう一様スケール、中心を原点に移動 (y も同じ倍率で拡張)
<code>Terrain::BakeHeightMap</code>	Terrain	メッシュを真上から高さマップに焼き込む。各三角形を xz 平面に投影し、内部の格子点で重心座標から y を補間、既存値より高ければ更新する (= 上から見える面)。メッシュに覆われない格子点は、覆われた隣接点の平均で順次埋める。
<code>Terrain::LoadTerrainConfig</code>	Terrain	assets/terrain.cfg (key=value 形式) を読み、投入位置の指定を取り込む 対応キー: emit_x, emit_z (プログラム内のワールド座標 [m], -30~30)
<code>Terrain::BuildHeightMap</code>	Terrain	GRID_N × GRID_N の格子点すべてで HeightFunction を評価し m_heights に保存

<code>Terrain::HeightFunction</code>	Terrain	地形の高さを手続的に計算する (高さマップの元になる関数)
<code>Terrain::BuildMesh</code>	Terrain	高さマップから頂点・インデックスを作り GPU バッファにする (手続き生成用)
<code>Terrain::CreateGpuBuffers</code>	Terrain	頂点配列とインデックス配列から IMMUTABLE な GPU バッファを作る
<code>Terrain::CreateRockTexture</code>	Terrain	岩肌テクスチャを用意する。assets フォルダの画像ファイル (ROCK_TEXTURE_FILES の順に探す) を読み込み、見つからない/読めない場合はノイズによる手続き生成に切り替える。
<code>Terrain::CreateProceduralRockTexture</code>	Terrain	ノイズから岩肌風のテクスチャを CPU で生成し、GPU テクスチャにする (画像ファイルがないときのフォールバック)
<code>Terrain::CreateShaders</code>	Terrain	Terrain.hlsl をコンパイルし、頂点/ピクセルシェーダーと入力レイアウトを作る
<code>Terrain::Render</code>	Terrain	地形メッシュを描画する (定数バッファ b0 は呼び出し側で設定済みとする)
<code>Terrain::GetHeight</code>	Terrain	任意の世界座標 (x, z) における地面の高さを双線形補間で返す
<code>Terrain::GetNormal</code>	Terrain	任意の世界座標 (x, z) における地面の法線 (単位ベクトル) を返す 中心差分で $\partial h / \partial x$, $\partial h / \partial z$ を求め $n = \text{normalize}(-h_x, 1, -h_z)$ とする
<code>Terrain::ValleyCenterX</code>	Terrain	手続き生成地形の谷の中心線の x 座標を z の関数として返す (蛇行カーブ)
<code>Terrain::GetValleyCenterX</code>	Terrain	現在の地形における谷の中心 x を返す (エミッタの投入位置決定用)。手続き生成地形なら ValleyCenterX の式、OBJ 地形ならその z の行で高さが最も低い x を高さマップから探す。
<code>Terrain::GetEmitPoint</code>	Terrain	粒子の投入位置 (x, z) を返す。assets/terrain.cfg に emit_x / emit_z があればそれを使い、なければ z = defaultZ の行で最も低い x (GetValleyCenterX) を使う。
<code>SPH::Initialize</code>	SPH	パラメータを設定し、配列を最大粒子数分だけ確保する
<code>SPH::Reset</code>	SPH	全粒子を消去し、経過時間とエミッタを初期状態に戻す
<code>SPH::Update</code>	SPH	実時間 frameTime 秒ぶんシミュレーションを進める。内部では timeStep 刻みのサブステップに分割して Step を繰り返す。
<code>SPH::Step</code>	SPH	1 サブステップぶん進める 手順: エミッタ → 近傍格子構築 → 密度/圧力 → 力 → 積分+衝突 → 排水
<code>SPH::Emit</code>	SPH	投入タイマーを進め、必要な回数だけ EmitSheet を呼ぶ
<code>SPH::EmitSheet</code>	SPH	上流の谷底に emitWidth × emitHeight の粒子の板を 1 枚置く
<code>SPH::BuildGrid</code>	SPH	全粒子をセルに振り分ける (カウンティングソート)。結果: m_cellStart[key] ~ m_cellStart[key+1] の範囲の m_sorted[] がハッシュ key のセルに属する粒子番号。
<code>SPH::CellCoord</code>	SPH	位置から格子セルの整数座標を求める ($\text{floor}(p / h)$)
<code>SPH::HashCell</code>	SPH	セル座標をハッシュ表の添え字 (0 ~ tableSize-1) に写像する 3 つの大きな素数との積の XOR を取る (Teschner et al., 2003)
<code>SPH::ForEachNeighbor</code>	SPH	粒子 i の周囲 27 セルを走査し、距離 < h の粒子 j ごとに func(j, dx, dy, dz, r2) を呼ぶ (dx.. は x_i - x_j, r2 はその 2 乗長) ハッシュの衝突で別セルの粒子が混ざらないよう、セル座標の一致も確認する

SPH::BuildNeighborLists	SPH	各粒子について距離 < h の近傍粒子番号を m_neighbors に記録する (並列)。密度計算と力計算の両方で同じ近傍を使うため、探索を 1 回で済ませる。
SPH::ComputeDensityPressure	SPH	各粒子の密度 ρ と圧力 p を計算する (並列)
SPH::ComputeForces	SPH	圧力力・粘性力・重力から各粒子の加速度を計算する (並列)
SPH::Integrate	SPH	シンプレクティック・オイラー法で速度と位置を更新し、境界と衝突させる (並列) $v \leftarrow v + a \, dt$, $x \leftarrow x + v \, dt$
SPH::ResolveCollision	SPH	粒子 i を地形と側壁に対して衝突処理する 地面より下なら地面の上に押し戻し、法線方向の速度成分を除去する
SPH::RemoveDrained	SPH	下流の端に達した粒子を配列から取り除く (末尾と入れ替えて詰める)
SPH::GetRenderData	SPH	描画用に (x, y, z, 速さ) を詰めた配列を返す。速さは近傍粒子との平均を取って平滑化する (SPH 特有の細かな速度の ばらつきで色がまだらにならないようにするため)。
FluidRenderer::Initialize	FluidRenderer	シェーダー・粒子バッファ・深度テクスチャを作成する
FluidRenderer::CreateShaders	FluidRenderer	Particle.hlsl / FluidSurface.hlsl をコンパイルしてシェーダーを作る
FluidRenderer::CreateDepthTargets	FluidRenderer	画面サイズの R32_FLOAT テクスチャを 2 枚 (ピンポン用) 作る
FluidRenderer::OnResize	FluidRenderer	画面サイズに依存する深度テクスチャを作り直す
FluidRenderer::UploadParticles	FluidRenderer	粒子配列を StructuredBuffer にコピーする
FluidRenderer::Render	FluidRenderer	粒子を指定モードで描画する。地形の描画後 (バックバッファと深度バッファが有効な状態) に呼ぶこと。
FluidRenderer::RenderParticlesMode	FluidRenderer	各表示モードの描画本体
FluidRenderer::RenderSurfaceMode	FluidRenderer	各表示モードの描画本体
FluidRenderer::DrawSpheres	FluidRenderer	現在バインドされているレンダーターゲットに粒子の球を描く
FluidRenderer::DrawFullscreen	FluidRenderer	フルスクリーン三角形を 1 枚描く (ぼかし・合成パス用)
Camera::Rotate	Camera	マウスドラッグ量に応じて yaw / pitch を回転させる
Camera::Pan	Camera	注視点をカメラの右方向・上方向に平行移動する (右ドラッグ用)
Camera::Zoom	Camera	注視点との距離を倍率で変える (マウスホイール用)
Camera::GetPosition	Camera	現在のカメラのワールド座標を返す
Camera::GetViewMatrix	Camera	左手系のビュー行列 (XMMatrixLookAtLH) を返す
Camera::GetProjMatrix	Camera	透視投影行列 (XMMatrixPerspectiveFovLH) を返す
MeshLoader::LoadObj	MeshLoader	OBJ ファイルを読み込み、三角形リストのメッシュにする

MeshLoader::ComputeSmoothNormals	MeshLoader	各三角形の面法線を頂点に加算して正規化し、滑らかな頂点法線を作る (面積の大きい三角形ほど強く寄与する重み付き平均になる)
MeshLoader::ComputeBounds	MeshLoader	全頂点からバウンディングボックス (boundsMin / boundsMax) を計算する
TextureLoader::LoadFromFile	TextureLoader	画像ファイルを読み込み、ミップマップ付きテクスチャの SRV を作る
TextureLoader::CreateFromPixels	TextureLoader	メモリ上の RGBA8 ピクセル配列からミップマップ付きテクスチャの SRV を作る (ファイル読み込みと手続き生成の両方が共通で使う)
Noise::Perlin2D	Noise	2 次元パーリンノイズを 1 点だけ評価する
Noise::Fbm2D	Noise	Perlin2D を複数オクターブ重ね合わせた fBm ノイズ
Noise::Ridged2D	Noise	「1 - noise 」を重ね合わせた尾根状ノイズ。岩の亀裂や山の稜線のような鋭い筋を作るのに向いている。
Noise::Perlin2DPeriodic	Noise	格子座標を period で折り返す「周期的」パーリンノイズ。x, y が period の整数倍ずれても同じ値になるため、継ぎ目なく繰り返せる (タイルング可能な) テクスチャに使える。
Noise::FbmTileable	Noise	タイルング可能な fBm。u, v ∈ [0,1) を入力とし、各オクターブで period を倍にしながら Perlin2DPeriodic を重ねる。
Noise::RidgedTileable	Noise	タイルング可能な尾根状ノイズ (岩の亀裂表現用)
Common.hlsl::JetColor	Common.hlsl	0～1 の値を「青 → 水色 → 緑 → 黄 → 赤」のグラデーションに変換する (参考動画の速度による着色を再現)
Common.hlsl::VSFullscreen	Common.hlsl	頂点バッファなしで画面全体を覆う 1 枚の三角形を生成する vid=0: (-1, 1), vid=1: (3, 1), vid=2: (-1, -3) → 画面全体を覆う
Terrain.hlsl::VSMain	Terrain.hlsl	頂点をクリップ空間へ変換し、法線と UV をそのまま渡す (地形のワールド行列は単位行列なので変換不要)
Terrain.hlsl::PSMain	Terrain.hlsl	テクスチャ色 × (環境光 + 拡散反射) で色を決める。さらにカメラから遠い谷底ほど少し暗くして奥行き感を出す。
Particle.hlsl::VSMain	Particle.hlsl	粒子 iid の中心をビュー空間に変換し、頂点 vid に対応する四角形の角を出力する
Particle.hlsl::SphereSurface	Particle.hlsl	四角形内の座標から球面上の点と法線を求める (共通処理)
Particle.hlsl::PSColor	Particle.hlsl	速度で着色した球を描く。拡散反射 + 鏡面反射でつやのある見た目にする。
Particle.hlsl::PSDepth	Particle.hlsl	表面生成用に球面のビュー空間 z を書き出す。深度テストにより地形の後ろや他の球の後ろは自動的に隠れる。
FluidSurface.hlsl::PSBlur	FluidSurface.hlsl	1 方向 (Params.zw で指定) のバイラテラルぼかし アルゴリズム: radiusPx = ワールド半径を画面に投影したピクセル数 (近いほど大きい) 重み $w = \exp(-k^2 / 2\sigma^2) * \exp(-(d_k - d_0)^2 / 2\sigma d^2)$ 出力 = $\sum w d_k / \sum w$
FluidSurface.hlsl::ViewPosFromDepth	FluidSurface.hlsl	ピクセル座標と深度からビュー空間座標を復元する $NDC\ x = x_view * Proj_m00 / z_view \rightarrow x_view = ndc.x * z / Proj_m00$
FluidSurface.hlsl::PSComposite	FluidSurface.hlsl	平滑化された深度から法線を求めて水面をシェーディングし、アルファブレンドで地形の上に描く。法線の求め方: 右隣・左隣との位置差 ddx, 上隣・下隣との位置差 ddy を計算し、輪郭 (深度が飛び場所) を避けるため差の小さい方を採用して $n = \text{normalize}(\text{cross}(\text{ddx}, \text{ddy}))$ とする。